

无人机 AI 算法研发计划(学生执行版)

制订:2026-07-30(v3 · 拆分细化版)| 负责人:聂方华 | 执行人:6-10 名学生(每人独立一题)

四大场景:灭火 · 搜救 · 巡检 · 快递配送

黄金范例: [path-ai](#) —— 把 path-ai 的成功范式复制到一批算法上

导读(先看这里)

这是什么	27 个无人机 AI 算法的「研发作战手册」:每人领 1 题,13 周内拿出 MVP(可调用函数)+论文/专利
给谁看	执行学生(照着做)/ 评审(对照查)/ 负责人(管进度)
怎么用	W1 先读 ★ 选题铁律 定题 → §三 认领 → §四 照卡片做 → §六 按周交 → §八 套模板
核心数字	4 场景 · 27 算法 · 6-10 人 · 13 周 · 服务端口 9204/9205/...(沿用 path-ai 9203 的 92XX)
黄金范例	path-ai :纯函数 <code>plan_route()</code> + 23 单测 + 独立 HTTP 服务 + README/CLAUDE/005 文档,每个算法照此复刻

O、path-ai 范式(所有算法的复刻模板)

[path-ai](#) 已跑通一条可复制路线:

明确场景 → 核心算法压成 1 个纯函数 `plan_route(req, cfg) -> Result<Resp>`
→ 配单测(正例/退化/边界)→ 包成独立 HTTP 服务(端口 9203)
→ DTO 对齐 `carrier` 契约 → README + 调用文档 → 专利交底书素材

三条铁律,所有算法必须遵守:

- 核心是一个可独立调用的纯函数(数据 + 配置 → 结果),不依赖网络/数据库/全局状态 —— 这是能申请专利、能写论文、能单测的前提。
- 几何/规则/公式类算法全手写、不引重型依赖(path-ai 刻意不用 `geo crate`)—— 流程步骤清晰、可画成专利权利要求、可做消融实验。
- Python 出论文,Rust 进平台,ONNX 桥接(详见 §五)—— 两阶段不割裂。

🔗 规划类算法是本团队强项(path-ai 即航线规划),也是最容易出专利的方向,选题优先向规划类(A/D/E)倾斜。

★ 选题与叙事铁律(最高优先级,凌驾所有规则之上)

一篇好论文 = 一个谁都能看懂的简单问题 × 一个有技术深度的困难解法。

「一眼看不出在研究啥」的题,选题评审直接打回。

读者(审稿人/评委)必须 **10 秒内** 看懂两件事:

- 1. **你在解决什么** —— 一句话场景,小白都懂(例:"送快递的无人机怎么排路线最省电")。
- 2. **为什么这事儿难** —— 不是"我用了 A* 算法",而是"它有具体的坎"。

困难的三种来源(贡献叙事,至少占一条)

难度来源	叙事套路	标题句式	贡献点
① 算法难	现有方法精度不够/做不到,我们设计新算法	"面向 X 的 Y 方法"	新模型 / 新公式 / 新流程
② 条件多	实际有 N 个约束,一项项建模、逐步满足	"考虑 A/B/C 约束的 X"	约束建模 + 渐进求解
③ 速度慢	精度够但跑不动实时,我们优化/加速	"实时 X 的轻量化 Y"	速度提升 × 倍,精度不降

精髓:事情很简单(能解决场景痛点),但算法很难 / 条件很多 / 速度很慢 —— 三者必居其一,论文才有技术深度。

好选题 vs 坏选题(对照)

	✗ 反例(空泛)	☑ 正例(一眼懂 + 难度明确)
规划	无人机路径规划研究	考虑载重·电量·时间窗的无人机配送路线优化(②)
灭火	基于深度学习的火灾检测	双光融合的远距离火点定位:把 50m 误差压到 5m(①)
速度	高效的目标检测算法	机载火点检测从 80ms/帧压到 10ms(③)

27 题卖点速查表(选题卡直接抄这一行定位)

编号	一句话简单问题(小白懂)	难度来源	场景
A1	来了个活儿,派哪架机、哪种机型最合适	② 载重+续航+载荷+距离+均衡	全部
A2	一片区域地毯式搜一遍别漏	② 覆盖率+电量+优先方向	搜救/巡检/灭火
A3	一批快递/巡检点,先送哪后送哪最省	② 时间窗+载重+电量(TSP/VRP)	配送/巡检
A4	飞着飞着情况变了,航线赶紧改	② 事件驱动+增量+电量	全部
A5	货越重越费电,重载航线怎么算才飞得到	① 载重-能耗耦合模型	配送/灭火
A6	一大块区域几架机分着扫,谁别累别留盲区	② 电量均衡+不重叠	搜救/灭火/巡检
B7	哪里着火了、火多大	①+③ 双光融合+实时	灭火
B8	废墟/野外里有没有人	① 小目标+低虚警	搜救
B9	电线杆/线缆坏没坏	① 小缺陷检测	巡检
B10	包裹送到哪个阳台/停机坪,准不准	② 安全+清晰+无遮挡	配送
B11	哪些车停在了不该停的地方	② 检测+GIS规则+去误报	巡检
B12	两期图比一比,哪里多出了新建筑	① 配准+变化检测	巡检
B13	这栋房子裂没裂、危不危险	① 裂缝细+样本少	巡检
C14	这火一会儿往哪烧	① 风+地形+植被多因子	灭火
C15	锁住那个人,带地面队去救	① 遮挡+引导	搜救
C16	飞着遇建筑/塔吊,实时局部绕开	③ 实时性	配送/全部
D17	从天上扔东西,提前多少、偏多少才能落准	① 风+载重弹道	灭火/配送/搜救
D18	一堆订单一堆机,怎么派最快	② 时间窗+充电+分配	配送
E19	几十架机航线交叉,怎么调高度/时间让它们不打架	②+① 分层错峰+NP-hard	灭火/搜救
E20	飞着突然有架机冲过来,毫秒级算出怎么躲	③ 实时感知-决策	灭火/搜救/全部
E21	禁飞区会变(火场扩散/管制),航线实时绕开还绕最短	② 时空禁飞区	灭火/全部
E22	灭火分侦察-压制-清理,每阶段派谁、几点飞	② 阶段+时段+机型+电量	灭火/搜救
E23	把天空划成时空走廊分给各机,结构上免冲突	② 4D 走廊分配	灭火/搜救/配送
E24	一群机摆阵型一起飞,队形保持还能变	① 编队动力学	搜救/灭火

编号	一句话简单问题(小白懂)	难度来源	场景
F25	取货送货配对,取货得先于送货,路线怎么排	② 取送优先序+容量+电量	配送
F26	几个基地一堆送货点,怎么分配+排路线最省	② 基地分配+电量均衡	配送
F27	正送着货来新急单,怎么临时插进当前航线	②+③ 在线插入+实时	配送

对写作的硬要求

- **标题** 必须含「问题域 + 方法/难度」,禁止只写"基于 XX 的 XX 研究"。
- **摘要第 1 句** = 那句简单问题,第 2 句 = 为什么难。10 秒入戏。
- **Introduction 的贡献点** 必须逐条对得上勾选的难度来源(①/②/③)。

一、总体目标与三阶段节奏

阶段	时间	目标	交付物
一·思路周	W0-1(第 1 周)	选题卡 + MVP 函数签名 + 单测骨架	选题卡(\$\wedge\$.1)+ 可编译空函数 + \$\geq\$3 单测
二·初稿	W2-9(月 1-2)	Python 跑通出指标 \$\rightarrow\$ Rust 落地 \$\rightarrow\$ 专利交底书/论文框架	Python 实验 + 指标表 + Rust crate/服务 + 文档 + 交底书初稿
三·实验发文	W10-13(月 3)	对比/消融实验 \$\rightarrow\$ 论文成稿/专利定稿	论文初稿 或 专利申请文件

硬节点: W1 末选题评审(定人定题定产出类型)· W5 末月 1 中检(Python 必须跑通)· **W9 末初稿 Deadline**(Rust 落地+交底书/论文框架)· W13 末定稿(可投递/申报)。

二、四大业务场景

#	场景	核心痛点	主要算法需求
S1	灭火	火点定位难、投弹精度差、火势蔓延不可测、大量机同空域易撞、分阶段展开	火点检测定位(B7)、火势预测(C14)、投弹落点(D17)、重载航线(A5)、分阶段调度(E22)、多机防撞(E20)、禁飞/危险区绕行(E21)、航迹解冲突(E19)
S2	搜救	大区域找人难、目标小易丢、引导地面队难、分时段大量机协同	覆盖搜索(A2)、人员检测(B8)、目标跟踪引导(C15)、物资投放(D17)、分区搜索(A6)、编队搜索(E24)、分时段调度(E22)、多机防撞(E20)
S3	巡检	线路长点多、缺陷小、城市违停/违建/危房多	多点路线(A3)、覆盖巡查(A2)、缺陷检测(B9)、违停检测(B11)、违建变化(B12)、危房检测(B13)、动态重规划(A4)
S4	快递配送	订单多、载重影响航程、投递精准、禁飞避障、取送/多基地/动态插单	配送投放路线(A3)、载重-航程优化(A5)、投递点识别(B10)、订单调度(D18)、障碍规避(C16)、任务分配(A1)、取送PDP(F25)、多车场(F26)、动态插单(F27)

算法协作数据流(你的算法在哪一环)



编号	算法	类别	场景	产出	难度
A1	无人机任务选派与机型匹配	规划·Rust	全部	📄 专利	★★★★
A2	区域覆盖搜索航线(弓字形/扩展方格)	规划·Rust	S2/S3/S1	📄 专利	★★★★
A3	多目标访问路线规划(TSP/VRP)	规划·Rust	S4/S3	📄 专利	★★★★★
A4	事件驱动动态重规划	规划·Rust	全部	📄 专利	★★★★
A5	载重-电量-航程联合优化	规划·Rust	S4/S1	📄 专利	★★★★★
A6	多机协同分区规划	规划·Rust	S2/S1/S3	📄 专利	★★★★★
B7	火点检测与定位(双光融合)	感知 ·Py→Rs	S1	📄 论文	★★★★
B8	受困人员检测(热成像)	感知 ·Py→Rs	S2	📄 论文	★★★★★
B9	输电线路/设备缺陷检测	感知 ·Py→Rs	S3	📄 论文	★★★★
B10	投递点识别与安全定位	感知 ·Py→Rs	S4	📄/📄	★★★★
B11	违停车辆检测	感知 ·Py→Rs	S3	📄 专利	★★★★
B12	违章建筑/新增建筑变化检测	感知 ·Py→Rs	S3	📄 论文	★★★★★
B13	危房结构损伤检测	感知 ·Py→Rs	S3	📄 论文	★★★★
C14	火势蔓延预测	预测 ·Py→Rs	S1	📄 论文	★★★★★
C15	目标跟踪与地面引导	跟踪 ·Py→Rs	S2	📄 论文	★★★★★
C16	动态障碍物局部规避	规划·Rust	S4/全部	📄 专利	★★★★
D17	投放/投弹落点解算	决策·Rust	S1/S4/S2	📄 专利	★★★★
D18	订单-无人机批量调度	决策·Rust	S4	📄 专利	★★★★★

编号	算法	类别	场景	产出	难度
E19	多机航迹解冲突(Deconfliction)	多机·Rust	S1/S2	📄 专利	★★★★★
E20	实时碰撞检测与避让(DAA)	多机·Py→Rs	S1/S2/全部	📄/📄	★★★★★
E21	动态禁飞区/危险区绕行重规划	多机·Rust	S1/全部	📄 专利	★★★★
E22	分阶段分时段任务编排	多机·Rust	S1/S2	📄 专利	★★★★★
E23	多机时空走廊分配(UTM)	多机·Rust	S1/S2/S4	📄 专利	★★★★★
E24	多机编队飞行控制	多机·Py→Rs	S2/S1	📄/📄	★★★★
F25	取送货一体化路径规划(PDP)	规划·Rust	S4	📄 专利	★★★★★
F26	多车场配送路径规划(MDVRP)	规划·Rust	S4	📄 专利	★★★★★
F27	飞行中动态插单路径调整	规划·Rust	S4	📄 专利	★★★★

📄 专利导向 / 📄 论文导向 / ★ 越多工作量越大。规划·决策·多机·配送路径类(A/C16/D/E/F)走 Rust 同栈、易出专利;感知·跟踪·预测类(B/C14/C15)走 Python、易出论文。最终产出以 §七 判定为准。

四、算法 MVP 定义卡

详细卡(A1/A3/B7/D17/E19)是模板,其余为精简卡。每张卡都给到「函数签名 + 数据集 + 指标 + 创新点 + MVP 边界」,让学生 W1 就能动手。

精简卡统一 7 字段:一句话 / 输入→输出 / 函数签名 / 数据集 / 评价指标 / 创新点 / MVP 边界。

📌 规划类(Rust 同栈,延续 path-ai 手写几何/图算法)—— 团队强项,优先选题

A1·无人机任务选派与机型匹配【详细卡·模板】

项	内容
一句话问题	来了个活儿(灭火/送货/搜救),机队里派哪架机、选哪种机型最合适
场景	全场景调度入口:异构机队(重载灭火机/轻载配送机/长续航侦察机)+ 任务队列
问题	任务对载重/续航/载荷(云台/灭火弹/物资舱)要求不同;纯"最近空闲"分配会大机干小活、小机干不了大活
输入	task (载重需求/航程/载荷类型/紧急度)+ fleet (各机机型/电量/位置/载荷挂载/可用性)
输出	Assignment { drone_id, model, reason, feasibility } + 不可行时的缺项说明
Python 验证签名	<pre>def assign_drone(task: Task, fleet: list[Drone]) -> Assignment</pre>
Rust 落地签名	<pre>pub fn assign_drone(req: &AssignReq, cfg: &Defaults) -> Result<AssignResp, ApiError></pre>
数据集/输入源	自建异构机队仿真(机型参数表 + 任务流生成器)
评价指标	分配成功率、机型-任务匹配正确率、机队负载均衡度(方差)、平均响应距离
创新点假说	多约束(载重/续航/载荷/距离)硬过滤 + 负载均衡软优化的两级选派方法 → 专利
难度来源	② 条件多:载重+续航+载荷+距离+机队均衡
MVP 边界	单任务分配、静态机队状态;不做多任务联合调度(那是 D18)

A2 · 区域覆盖搜索航线【精简卡】

- **一句话**:一片区域地毯式搜一遍别漏。path-ai 弓字形的搜索版,新增「扩展方格/螺旋搜索」(搜救标准模式)。
- **输入→输出**: 区域 + 起点 + 电量 + 搜索优先方向 → 覆盖搜索航线 + 覆盖率/电量统计。
- **签名**: `fn coverage_search(req, cfg) -> Result<RoutePlanResp, ApiError>`。
- **数据集**:自建区域多边形 + 公开巡查/搜索仿真场景。
- **评价指标**:覆盖率 %、重复率 %、总距离、预估耗电。
- **创新点**:扩展方格搜索 + 电量感知的搜索优先级(电量不足先搜下风向/高概率区)→ 专利。
- **MVP 边界**:矩形区域弓字形 + 螺旋;不做人形先验概率图。

A3 · 多目标访问路线规划(TSP/VRP)【详细卡·模板】(配送投放最优路径核心)

项	内容
一句话问题	一批快递投递点/巡检塔,先去哪后去哪最省电、还能赶得上时间窗
场景	配送(多点投递,快递投放最优路径)、巡检(多塔依次巡检)
问题	经典 TSP/VRP;无人机多了电量约束、载重衰减、时间窗,比车辆 VRP 更难
输入	targets (N 点 + 各自时间窗/需求量)+ home + 电量/载重
输出	访问顺序航线 + 总距离/电量/是否满足时间窗
Python 验证签名	<pre>def plan_multistop(targets: list[Target], home, battery, payload) -> RoutePlan</pre>
Rust 落地签名	<pre>pub fn plan_multistop(req: &MultiStopReq, cfg: &Defaults) -> Result<RoutePlanResp, ApiError></pre>
数据集	Solomon VRPTW 基准、自建无人机配送场景
评价指标	总距离、按时送达率、电量可行性、与 OR-Tools 最优解的 gap
创新点假说	电量+时间窗+载重三约束的无人机 VRP 启发式(构造 + 局部搜索)→ 专利
难度来源	② 条件多;NP-hard,大规模要近似
MVP 边界	≤20 点、单机、仅电量约束;时间窗作为进阶

A4 · 事件驱动动态重规划【精简卡】

- 一句话:飞着飞着火势变了/来新订单/电量告警,航线赶紧改。
- 输入→输出: 当前状态 + 剩余电量 + 触发事件 → 新航线。
- 签名: `fn replan(req, cfg) -> Result<RoutePlanResp, ApiError>`。
- 数据集:自建事件流仿真(电量/任务/环境突变)。
- 评价指标:重算耗时、新旧航线偏差、增量 vs 全量重算加速比。
- 创新点:不全量重算的增量重规划(保留已飞段,只重算受影响段)→ 专利。
- MVP 边界:电量阈值触发直线返航。

A5 · 载重-电量-航程联合优化【精简卡】

- 一句话:货越重越费电,重载航线怎么算才飞得到。
- 输入→输出: 载重-耗电曲线 + 航点 + 电量 → 计入载重衰减的可行航线。
- 签名: `fn plan_heavy(req, cfg) -> Result<RoutePlanResp, ApiError>`。
- 数据集:机型载重-耗电参数表 + 投放/配送航点。
- 评价指标:航程可行性判定准确率、对比忽略载重方案的坠机/返航率。
- 创新点:载重-能耗耦合的航程可行性模型(path-ai 电量预算的载重增强版)→ 专利。
- MVP 边界:线性载重-耗电模型。

A6 · 多机协同分区规划【精简卡】

- 一句话:一大块区域几架机分着扫,谁都别累、别留盲区。
- 输入→输出: 大区域 + N 机 + 各机电量 → N 个子区域 + 各自航线。
- 签名: `fn partition(req, cfg) -> Result<MultiPlanResp, ApiError>`。
- 数据集:自建大区域多边形 + 异构机队。
- 评价指标:覆盖率、负载均衡度(各机工作量方差)、子区重叠率。
- 创新点:电量 + 覆盖均衡的子区域划分 → 专利。

- MVP 边界:矩形区域均分 + 电量加权。

B 感知检测类(Python 训练 → ONNX → Rust 推理)

城市巡检的「违停 / 违建 / 危房」已按物理本质拆为三个独立方向(B11/B12/B13):违停=检测+规则(专利),违建=多期变化检测(论文),危房=结构损伤分类(论文),数据集与产出类型均不同,各自独立成题。

B7 · 火点检测与定位(双光融合)【详细卡·模板】

项	内容
一句话问题	从空中看清哪里着火了、火多大、能不能精准定位坐标
场景	S1 灭火侦察:可见光看烟/火,热成像穿透烟雾看火核
问题	单可见光被烟雾遮挡、远距离火点小;单热成像虚警高(发热非火目标);需双光融合 + 定位
输入	可见光帧 frame_rgb + 热成像帧 frame_thermal + 相机姿态(定位解算用)
输出	<code>list[FireSpot { bbox, heat_intensity, lon, lat, confidence }]</code>
Python 验证签名	<code>def detect_fire(rgb: np.ndarray, thermal: np.ndarray, pose: Pose) -> list[FireSpot]</code>
Rust 落地签名	<code>pub fn detect_fire(req: &FireDetectReq, cfg: &Defaults) -> Result<FireDetectResp, ApiError> (ort 推理)</code>
数据集	FASDD、FLAME、D-Fire;热成像需自建或公开热成像火灾集
评价指标	检测 mAP、定位误差(m)、远距离(>100m)检出率、虚警率、FPS
创新点假说	可见光+热成像双光融合的火点检测与像素-地理定位解算 → 论文(融合策略 + 定位精度对比)
难度来源	① 算法难(融合+定位)+ ③ 速度(机载实时)
MVP 边界	单帧、白天、已知相机姿态;不做视频时序

B8 · 受困人员检测(热成像)【精简卡】

- 一句话:废墟/野外里有没有人。热成像人形检测为主,可见光辅助。
- 输入→输出: 热成像帧(+可见光帧) → 人员框 + 置信度。
- 签名: `fn detect_person(req, cfg) -> Result<DetectResp, ApiError>`。
- 数据集:热成像行人、LLVIP、SAR(Search & Rescue)数据集。
- 评价指标:检出率、远距离小目标 mAP、虚警率。
- 创新点:复杂背景(树丛/废墟热斑)下的低虚警人员检测 → 论文。
- MVP 边界:热成像 YOLO 基线。

B9 · 输电线路/设备缺陷检测【精简卡】

- **一句话:**电线杆、线缆、绝缘子坏没坏。
- **输入→输出:** 航拍帧 → 缺陷框 + 类型(破损/断股/异物)。
- **签名:** `fn detect_defect(req, cfg) -> Result<DetectResp, ApiError>`。
- **数据集:**绝缘子缺陷集、TTPLA(输电线路提取)、PLDM。
- **评价指标:**缺陷 mAP、分类准确率。
- **创新点:**航拍小目标缺陷检测 → 论文。
- **MVP 边界:**绝缘子破损二分类。

B10 · 投递点识别与安全定位【精简卡】

- **一句话:**包裹送到哪个阳台/停机坪/投递柜,识别准不准、安不安全。
- **输入→输出:** 航拍帧 + 候选区域 → 投递点框 + 安全评分。
- **签名:** `fn locate_drop_site(req, cfg) -> Result<DropSiteResp, ApiError>`。
- **数据集:**自建航拍地标/投递点集(阳台·停机坪·投递柜分类)。
- **评价指标:**识别率、定位精度、安全判定准确率。
- **创新点:**面向投递的着陆点安全评估(无遮挡/无行人/平整)→ 专利/论文。
- **MVP 边界:**投递框识别 + 面积评分。

B11 · 违停车辆检测【精简卡】

- **一句话:**哪些车停在了不该停的地方(禁停区/压线/跨车位)。
- **输入→输出:** 航拍帧 + GIS 禁停区多边形 → 违停框 + 违规类型。
- **签名:** `fn detect_illegal_parking(req, cfg) -> Result<DetectResp, ApiError>`。
- **数据集:**CARPK、UAVDT(航拍车辆)+ 自建禁停区标注。
- **评价指标:**违停判定 mAP、误报率(合法车被判违停)、漏报率。
- **创新点:**「DNN 车辆检测 + GIS 禁停区空间约束 + 压线判定规则」三位一体方法 → **专利**(判定流程可画成权利要求)。
- **难度:**② 条件多。 **MVP 边界:**单帧、白天、已知禁停区多边形。

B12 · 违章建筑/新增建筑变化检测【精简卡】

- **一句话:**两期航拍图比一比,哪里多出了新建筑/违章搭建。
- **输入→输出:** 旧期影像 + 新期影像 → 变化掩膜 + 新增建筑框。
- **签名:** `fn detect_illegal_construction(req, cfg) -> Result<ChangeDetectResp, ApiError>`。
- **数据集:**LEVIR-CD、WHU building、SpaceNet(建筑/变化检测)。
- **评价指标:**变化 IoU、F1、OA(总体精度)。
- **创新点:**多期影像配准 + 语义变化网络 → 论文。
- **难度:**① 算法难(配准 + 变化,精度难提)。 **MVP 边界:**配准 + 差分阈值基线。

B13 · 危房结构损伤检测【精简卡】

- **一句话:**这栋房子裂没裂、倾斜没、危不危险。
 - **输入→输出:** 建筑特写帧 → 损伤类型(裂缝/倾斜/剥落)+ 严重度分级。
 - **签名:** `fn detect_structural_damage(req, cfg) -> Result<DamageResp, ApiError>`。
 - **数据集:**SDNET2018、CRACK500(裂缝);倾斜姿态需自建。
 - **评价指标:**裂缝 IoU、损伤分类准确率、严重度分级一致率。
 - **创新点:**裂缝语义分割 + 倾斜姿态估计联合的危房分级 → 论文。
 - **难度:**① 算法难(裂缝细、样本少)。 **MVP 边界:**裂缝二分类。
-

📌 跟踪 / 预测类(Python → ONNX → Rust)

C14 · 火势蔓延预测【精简卡】

- 一句话:这火一会儿往哪烧、烧多大。
- 输入→输出: 火点 + 风速风向 + 地形 + 植被 + 时间 → 未来 N 分钟火势范围。
- 签名: `fn predict_fire(req, cfg) -> Result<FireSpreadResp, ApiError>`。
- 数据集:FARSITE 模拟数据、FWI 指数数据、历史火场记录。
- 评价指标:范围 IoU、蔓延方向误差、蔓延速率误差。
- 创新点:风+地形+植被多因子的火势蔓延模型 → 论文/专利。
- MVP 边界:风速主导的椭圆蔓延模型。

C15 · 目标跟踪与地面引导【精简卡】

- 一句话:锁住那个受困者/嫌疑人,带地面救援队过去。
- 输入→输出: 视频流 + 初始目标框 → 轨迹 + 引导指令(方位/距离)。
- 签名: `fn track_and_guide(req, cfg) -> Result<TrackGuideResp, ApiError>`。
- 数据集:VisDrone-MOT、SAR 跟踪集、GOT-10k。
- 评价指标:MOTA、IDF1、引导落点误差。
- 创新点:无人机视角跟踪 + 实时地面引导指令生成 → 论文。
- MVP 边界:单目标 SOT + 方位解算。

C16 · 动态障碍物局部规避【精简卡】

- 一句话:飞着遇建筑/塔吊等障碍物,实时局部绕开。
- 输入→输出: 航线 + 障碍多边形 + 当前位 → 规避后航线。
- 签名: `fn avoid_obstacle(req, cfg) -> Result<RoutePlanResp, ApiError>`。
- 数据集:自建障碍场景 + AirSim 城市环境。
- 评价指标:规避成功率、绕行距离增量、重算耗时(ms)。
- 创新点:实时局部重规划(A*/RRT 变体)→ 专利。
- 难度:③ 实时性。 MVP 边界:静态多边形障碍绕行。
- 边界区分:本题=单机对静态/动态障碍物局部避障;多机间实时防撞=E20、动态禁飞区绕行=E21、预规划解冲突=E19。

📌 决策/优化类

D17 · 投放/投弹落点解算【详细卡·模板】

项	内容
一句话问题	从空中扔灭火弹/包裹/救援物资,提前多少、偏多少才能正好落到目标点
场景	S1 灭火投弹、S4 配送投放、S2 搜救物资(三者物理本质相同:抛物 + 风 + 载重)
问题	高空投放受横风、投掷物自重/阻力、无人机航速影响,落点偏离大;靠人盯不准
输入	高度 + 航速 + 风速风向 + 投掷物参数(质量/阻力系数)+ 目标坐标
输出	投放点坐标 + 提前量 + 预测落点误差
Python 验证签名	<pre>def solve_drop_point(alt, speed, wind, payload, target) -> DropSolution</pre>
Rust 落地签名	<pre>pub fn solve_drop_point(req: &DropReq, cfg: &Defaults) -> Result<DropResp, ApiError></pre>
数据集	投放弹道仿真(自建,物理引擎/解析解生成)
评价指标	落点误差(m)、不同风速下的鲁棒性、解算耗时
创新点假说	风+载重+阻力耦合的投放弹道解算(解析 + 数值修正)→ 专利 (物理公式 + 解算流程,权利要求清晰)
难度来源	① 算法难(多物理量耦合弹道)
MVP 边界	无风 + 恒定航速 + 忽略阻力;风/阻力作为进阶

D18 · 订单-无人机批量调度【精简卡】

- **一句话**:一堆订单、一堆机,怎么派最快、还能按时、电量够、该充电充电。
- **输入→输出**: 订单批次(时间窗/重量)+ fleet + 充电站 → 调度方案(每机任务序列)。
- **签名**: `fn dispatch(req, cfg) -> Result<DispatchResp, ApiError>`。
- **数据集**:Solomon VRPTW、自建配送订单流。
- **评价指标**:完工时间、按时率、总能耗。
- **创新点**:带充电站访问 + 时间窗的无人机车队调度(VRPTW 变种)→ 专利。
- **MVP 边界**:≤30 订单、忽略充电。

多机协同与空域安全(分阶段·大量无人机,灭火/搜救重点)

灭火/搜救往往**分阶段、分时段**展开,动辄**几十架无人机**同空域作业 —— 既要避免相互碰撞,又要绕开动态禁飞区/火场危险区。本类是规模化落地的必答题,全是规划/决策型,**Rust 同栈、易出专利**。

E19 · 多机航迹解冲突(Trajectory Deconfliction)【详细卡·模板】

项	内容
一句话问题	几十架机的航线在空中交叉,怎么提前调高度/时间/速度,让它们互不相撞
场景	灭火/搜救大规模出动:各机各自规划了航线,合到一起发现轨迹交叉
问题	两两航迹冲突检测是 $O(N^2)$;解除冲突要兼顾电量/时间窗/任务优先级,是 NP-hard
输入	N 条已规划航迹(4D:经纬度+高度+时间)+ 安全间距
输出	解冲突后的航迹集(各机高度层/时间偏移/速度调整)+ 冲突报告
Python 验证签名	<pre>def deconflict(trajectories: list[Trajectory], sep: float) -> list[Trajectory]</pre>
Rust 落地签名	<pre>pub fn deconflict(req: &DeconflictReq, cfg: &Defaults) -> Result<DeconflictResp, ApiError></pre>
数据集/输入源	自建多机航迹仿真 + 公开 4D 轨迹冲突数据集
评价指标	残留冲突数(目标=0)、平均时间/高度偏移、电量影响、解算耗时 vs 机型数
创新点假说	「高度分层 + 时间错峰 + 优先级」的分级解冲突策略 → 专利 (流程清晰,权利要求好写)
难度来源	② 条件多 + ① NP-hard(大规模要近似)
MVP 边界	≤10 机、仅高度层调整;时间/速度调整作为进阶

E20 · 实时碰撞检测与避让(DAA, Detect And Avoid)【精简卡】

- **一句话:**飞着突然有架机/障碍冲过来,毫秒级算出怎么躲才不撞。
- **输入→输出:** 本机状态 + 临近目标轨迹(ADS-B/雷达/视觉) → 避让机动(速度/航向/高度变化量)。
- **签名:** `fn avoid_collision(req, cfg) -> Result<ManeuverResp, ApiError>`。
- **数据集:**自建接近场景仿真、UAV DAA 数据集。
- **评价指标:**碰撞避免率、最近会遇距离(CPA)、决策延迟(ms)。
- **创新点:**毫秒级实时感知-决策避让(预测 CPA + 机动解算)→ 论文/专利。
- **难度:**③ 实时性。 **MVP 边界:**2 机对飞避让。
- **边界区分:**本题=飞行中实时多机防撞闭环;预规划解冲突=E19、单机障碍规避=C16。

E21 · 动态禁飞区/危险区绕行重规划【精简卡】

- **一句话:**禁飞区会变(火场扩散/临时管制/他机安全包络),航线要实时绕开还绕得最短。
- **输入→输出:** 航线 + 动态禁飞多边形(带生效/失效时间) → 绕行后航线。
- **签名:** `fn reroute_nfz(req, cfg) -> Result<RoutePlanResp, ApiError>`。
- **数据集:**自建动态禁飞场景(火场扩散模型 + 管制区时变)。
- **评价指标:**绕行距离增量、安全性(全程在禁飞区外)、重算耗时。
- **创新点:**时空(4D)禁飞区感知的最短绕行重规划 → 专利。
- **难度:**② 条件多(带时效的多边形)。 **MVP 边界:**单个动态圆形禁飞区绕行。

E22 · 分阶段分时段任务编排(灭火/搜救阶段调度)【精简卡】

- **一句话:**灭火分侦察-压制-清理、搜救分黄金期-扩展期,每阶段派谁、几点起飞、飞多久。
- **输入→输出:** 任务阶段定义(各阶段目标/机型需求/时段)+ 机队状态 → 各阶段机群分配 + 时序甘特图。
- **签名:** `fn schedule_phases(req, cfg) -> Result<PhasePlanResp, ApiError>`。
- **数据集:**自建分阶段任务剧本(灭火/搜救标准流程)。
- **评价指标:**阶段衔接平滑度、总完工时间、机队利用率。
- **创新点:**阶段依赖 + 时段 + 机型 + 电量四约束的阶段编排 → 专利。
- **难度:**② 条件多。MVP 边界:2 阶段、静态机队。

E23 · 多机时空走廊分配(UTM Corridor)【精简卡】

- **一句话:**把天空划成一条条时空走廊分给各机,从结构上让它们打不起来。
- **输入→输出:** 机队 + 任务 + 空域网格 + 时段 → 时空走廊(4D 管道)分配方案。
- **签名:** `fn assign_corridor(req, cfg) -> Result<CorridorResp, ApiError>`。
- **数据集:**自建空域网格仿真。
- **评价指标:**走廊冲突率、空域利用率、分配成功率。
- **创新点:**4D 时空走廊的冲突免分配(空域结构化)→ 专利。
- **难度:**② 条件多。MVP 边界:规则网格 + 单时段。

E24 · 多机编队飞行控制【精简卡】

- **一句话:**一群机摆成阵型一起飞(搜救地毯式、灭火齐射),队形还得保持住、能变换。
- **输入→输出:** 编队构型(各机相对位置)+ 长机轨迹 → 各僚机相对位置/速度指令。
- **签名:** `fn formation_control(req, cfg) -> Result<FormationResp, ApiError>`。
- **数据集:**AirSim/Gazebo 编队仿真。
- **评价指标:**队形保持误差、队形变换时间、抗扰动恢复时间。
- **创新点:**长机-僚机编队保持与变换控制律 → 论文/专利。
- **难度:**① 编队动力学。MVP 边界:3 机三角形编队直线飞行。

■ 配送路径专题(快递配送场景,A3/D18 的专门变种,全规划·Rust)

A3 解决「单机单基地、纯送货」的访问顺序;现实配送还有三类常见变体——**取送货配对**、**多基地**、**动态插单**——各自约束不同,独立成题更聚焦。全规划型,Rust 同栈、易出专利。

F25 · 取送货一体化路径规划(PDP)【精简卡】

- **一句话:**既要去 A 处取货、又要送到 B 处(取货必须先于送货),怎么排路线最省。
- **输入→输出:** 订单列表(每个含取点+送点+货物量)+ home + 电量/载重 → 满足「取先于送」的访问顺序航线。
- **签名:** `fn plan_pdp(req, cfg) -> Result<RoutePlanResp, ApiError>`。
- **数据集:**Li & Lim PDPTW 基准、自建取送货场景。
- **评价指标:**总距离、优先序违反数(目标=0)、载重峰值、与最优解 gap。
- **创新点:**载重随取送货动态变化 + 电量约束的 PDP 启发式 → 专利。
- **难度:**② 条件多(取送优先序 + 容量 + 电量)。MVP 边界:≤10 订单、单机、仅优先序约束。

F26 · 多车场配送路径规划(MDVRP)【精简卡】

- **一句话:**几个无人机基地各自有机、一堆送货点,怎么分配 + 排路线总体最省。
- **输入→输出:** 多基地 + 送货点 + 各基地机队/电量 → 基地-点分配 + 各基地航线。
- **签名:** `fn plan_multidepot(req, cfg) -> Result<MultiPlanResp, ApiError>`。
- **数据集:**Cordeau MDVRP 基准、自建多基站配送场景。

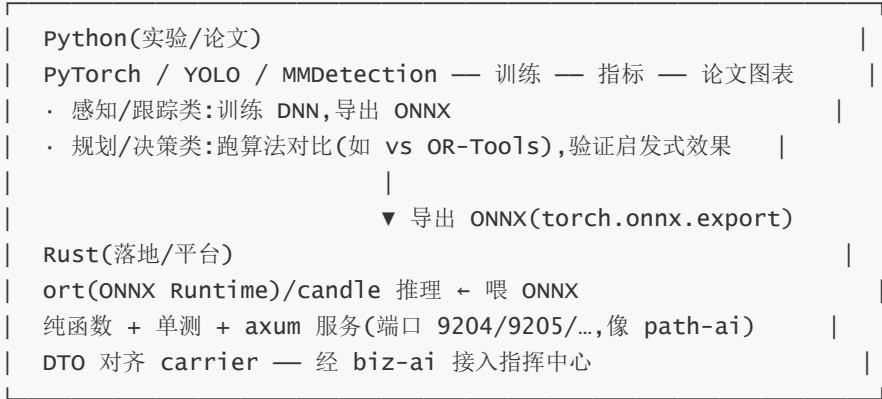
- **评价指标:**总距离、基地负载均衡度、电量可行性、与最优解 gap。
- **创新点:**基地分配 + 电量均衡的多车场 VRP → 专利。
- **难度:**② 条件多(分配 + 路线两层)。**MVP 边界:**2 基地、≤20 点。

F27 · 飞行中动态插单路径调整(Dynamic/Online VRP)【精简卡】

- **一句话:**无人机正送着货,突然来个新急单,怎么临时插进当前航线最划算。
- **输入→输出:** 当前执行航线 + 当前位置 + 新订单 + 剩余电量 → 插入新订单后的调整航线。
- **签名:** fn insert_order(req, cfg) -> Result<RoutePlanResp, ApiError>。
- **数据集:**自建动态订单流(随机到达 + 时间窗)。
- **评价指标:**绕路距离增量、时间窗满足率、插入决策延迟、电量可行性。
- **创新点:**在线插入式重排(局部邻域搜索 + 电量可行性增量校验,不全量重算)→ 专利。
- **难度:**② 条件多 + ③ 在线实时。**MVP 边界:**单机、单新订单插入。

五、技术栈分工:Python 验证 → Rust 落地

核心原则:Python 出论文,Rust 进平台,ONNX 是桥梁。



算法类别	Python 侧(论文)	Rust 侧(落地)	桥接
A 规划 / D 决策 / E 多机 / F 配送 路径(A1-A6,C16,D17,D18,E19-E24,F25-F27)	跑算法实验 (vs OR-Tools 等基线、消融、规模扩展)+ 可视化	主线 ,手写几何/图算法	一般不涉及 DNN,直接 Rust
B 感知 / C 跟踪预测(B7-B13,C14,C15)	PyTorch 训练 + 评测	ort 推理 ONNX + 前后处理	ONNX

⚠ **别误解:**规划类不是「不用 Python」。VRP/解冲突/调度这类 NP-hard 题,论文要达标**必须**在 Python 侧跑对比实验(对标 OR-Tools 最优解、做消融和规模扩展),Rust 只是最终工程落地。

约定:

- 每个 DNN 算法的 Python 仓库存模型 + 训练 + 评测 + `export_onnx.py`;Rust crate 只读 ONNX 推理。
- 规划/决策类与 path-ai 同构,最终交付 Rust;Python 用于算法验证与论文实验。
- Rust crate 放 `zhuque-ai-algorithm/<algo-name>/`,端口 9204、9205 ... 递增。

六、每周里程碑与交付物

周	阶段	每人交付物	评审重点
W0	选题	认领算法(S三)	不撞车、难度匹配
W1	思路周	选题卡(S八.1)+ 可编译空函数 + ≥3 单测	函数签名 + MVP 边界 + 卖点是否清晰
W2	初稿·Py	Python 数据管线 + 基线模型	数据集就绪、基线能出数
W3	初稿·Py	基线指标 + 错误分析	指标合理、瓶颈在哪
W4	初稿·Py	改进版 + 初步创新点验证	创新点是否有效
W5	月1中检	Python 跑通 + 指标表 + ONNX 导出	能否进 Rust;跑不通则简化
W6	初稿·Rs	Rust crate 骨架 + 推理跑通	Rust 与 Python 结果一致
W7	初稿·Rs	纯函数 + 单测(≥10) + axum 服务	对齐 path-ai 范式
W8	初稿·Doc	README + 调用文档 + 交底书/论文框架初稿	文档完整性
W9	初稿 Deadline	Rust 落地 + 文档 + 交底书/论文框架	初稿验收,定专利/论文去向
W10	实验	实验设计 + 对比基线选定	实验能证伪创新点
W11	实验	对比实验 + 消融实验	数据是否充分
W12	发文	论文/专利成稿	写作质量
W13	定稿	可投递论文 / 可申报专利	结题

七、专利 vs 论文判定标准

一句话:能写成「方法步骤 + 公式/结构」的 → 专利;创新在模型/数据/调参的 → 论文。

特征	倾向	本计划对应算法
有明确流程步骤、几何/图算法、判定规则、数学/物理公式	 专利 (方法发明专利)	A1-A6、C16、D17、D18、E19/E21/E22/E23、F25/F26/F27、B10、B11
创新在 DNN 结构、损失函数、数据驱动、需大量对比/消融	 论文	B7-B9、B12、B13、C14、C15
二者皆可(方法组合型)	先写 交底书 占坑,W9 评审定	B7、B10、C14、E20、E24

判定流程: W1 选题初判 → W9 初稿终判。**能专利的优先专利(锁定知识产权),论文作为进度允许的加分。**规划类是出专利的主力。

八、文档模板与评审

8.1 选题卡模板(W1 交,每人一份)

```
# <算法名> · 选题卡
> 作者:___ 算法编号:___ 场景:S_ 推荐产出: ☒ 专利/☒ 论文

## 1. 场景与问题(先填卖点,再写细节)
- **一句话简单问题(小白能懂,必填)**:___
- **难度来源(必填,勾选 ≥1,见 ★ 铁律)**:☐ ①算法难 ☐ ②条件多(约束:___) ☐ ③速度慢(现状 ___
→ 目标 ___)
- **一句话标题草稿**:_
- 详细:在什么场景下解决什么问题,为什么 path-ai / 现有方案不够。

## 2. MVP 定义
- 输入 / 输出:
- 核心函数签名(Python / Rust):
- MVP 边界(明确不做什么):

## 3. 数据集 / 输入来源
## 4. 评价指标
## 5. 创新点假说(1-3 条,每条标注 专利/论文 价值)
## 6. 技术路线(3 句话)
## 7. 风险与降级方案
```

8.2 专利交底书大纲(对应 path-ai 的 [005-巡视算法调用.md](#))

1. 背景与场景:现有技术缺陷
2. 发明目的:要解决什么
3. 技术方案(核心,对应 path-ai `plan_route` 9 步):**逐步骤 + 公式 + 流程图** ← 权利要求来源
4. 关键创新点:区别于现有技术
5. 实施例:测试数据/图
6. 有益效果:量化对比(对应 path-ai 的 stats)

8.3 论文大纲(IEEE / 中文核心通用)

Abstract · Introduction(动机+贡献) · Related Work · Method(对应 MVP 函数) · Experiments(数据集/指标/对比/消融) · Conclusion

8.4 完成定义 DoD(对齐 path-ai)

初稿「完成」须同时满足:

- ☐ 核心纯函数 + **≥10 单测**(正例/退化/边界,参照 path-ai 23 例)
- ☐ **README.md + CLAUDE.md**(参照 path-ai 两份文档)
- ☐ Python:可复现实验脚本 + 指标表
- ☐ Rust:独立 crate/服务(92XX 端口)+ DTO 对齐 carrier
- ☐ 专利交底书初稿 **或** 论文框架(含实验设计)

8.5 评审节奏

- 每周 1 次走查(每人 15-30 分钟):对照 path-ai 范式看「函数 + 单测 + 文档」。
- 4 个硬节点全员评审(W1 / W5 / W9 / W13):选题、中检、初稿、定稿。
- 结对范例:每个新算法的 README/CLAUDE.md 以 path-ai 两份文档为模板。

九、风险与对策

风险	对策
Python 与 Rust 结果不一致	W6 强制对齐:同输入两版输出 diff,误差进单测断言
DNN 算法在 Rust 推理卡壳	W9 前 Rust 可只做「前后处理 + 调 ONNX」;卡住则降级为「Python 服务 + Rust 客户端」过渡
数据集稀缺(火势/热成像/事故)	仿真合成(FARSITE/物理引擎)+ 数据增强;或降级到更易取数的子问题
学生进度差异大	27 题分难度 ★ 按水平分配;W5 中检不达标强制简化 MVP
多机算法(E19-E24)难验证	AirSim/Gazebo + 轻量自建多机仿真;先 2-3 机小规模再扩展
创新点不足以发文	每张卡「创新点假说」W1 即可证伪;W5 不成立则换创新角度
场景跨度过大(灭火到配送)	每人只做一个窄题;横切规划类(A)跨场景复用,避免重复造轮子

十、立即可执行的第一步

1. 本周内:把本计划发给学生,每人认领 1 个算法(§三),交选题卡(§八.1)。
2. W1 末:开选题评审会,定人定题定产出类型,同时定每人的 92XX 端口号。
3. 参照 [path-ai 源码](#) + [README.md](#) + [CLAUDE.md](#) + [005 文档](#)作为每个算法的「四件套」模板。